

Desarrollo ágil de software

Cuando hablamos de ingeniería de software debemos tener en claro que existen diferentes metodologías ágiles. Entre ellas, podemos mencionar a la metodología ágil *scrum,* la metodología ágil XP y la metodología ágil *kanban*. Cuando se desarrolla un proyecto de *software* es necesario conocer el desarrollo dirigido por un plan, la administración de un proyecto ágil y el escalamiento de un *software*.

Desarrollo ágil de software

Video conceptual

Referencias

Desarrollo ágil de software

Metodologías ágiles

La Empresa Sumar ya seleccionó la consultora para desarrollar el proyecto y ahora le surge la necesidad de conocer las metodologías ágiles que existen en la actualidad para identificar la metodología a utilizar. Necesita preparar el desarrollo de un plan, identificar todos los elementos necesarios para administrar un proyecto ágil y conocer los elementos que componen el escalamiento de un *software*.

Ante la situación planteada, la Empresa Sumar requiere asesoramiento para poder desarrollar cada uno de los proyectos. **A fin de aportarle soluciones, se le presentan elementos para que pueda administrar de forma adecuada un proyecto.**

Conversemos sobre proyectos

La gestión ágil de proyectos es un enfoque iterativo para planificar y guiar los procesos del proyecto. Al igual que en el desarrollo de *software* ágil, un proyecto ágil se completa en pequeñas secciones llamadas iteraciones.

Cada iteración es revisada y criticada por el equipo del proyecto, puede incluir a representantes de la empresa del cliente y empleados. Los conocimientos adquiridos a partir de la crítica de una iteración se utilizan para determinar cuál debe ser el siguiente paso en el proyecto. Cada iteración del proyecto está programada normalmente para ser completada en dos semanas.

El principal beneficio de la gestión de proyectos ágiles es su capacidad para responder a los problemas que pueden surgir a lo largo del transcurso del proyecto. Hacer un cambio necesario para un proyecto en el momento adecuado puede ahorrar recursos y en última instancia, ayudar a entregar un proyecto exitoso a tiempo y dentro del presupuesto.

No obstante, debido a que la gestión ágil se basa en la capacidad de tomar decisiones con rapidez, no es adecuada para las organizaciones que tienden a deliberar sobre cuestiones durante un período prolongado o llevan las decisiones a un comité (Scrum, s. f. a).

¿Cuáles son las herramientas utilizadas en la metodología ágil?

Las herramientas utilizadas en la metodología ágil se pueden usar de varias maneras. Esto va a depender de cómo se esté llevando a cabo el enfoque orientado de forma ágil. Las herramientas pueden incluir lo siguiente.

- Listas de trabajo pendiente priorizadas: permiten priorizar y volver a establecer prioridades entre las historias de usuario y los errores al arrastrar y soltar las tarjetas de registro.

- Tableros *kanban* o *scrum*: permiten ver todas las historias de usuarios con tarjetas que muestran las tareas, las cesiones y los estados en una iteración.

- Tareas o columnas de natación: separan cesionarios, proyectos, etc. al mover las tarjetas de arrastrar y soltar a través de los carriles.

- Flujos de trabajo: se pueden crear flujos de trabajo personalizados que actualicen los problemas automáticamente en función de eventos específicos.

- Iteraciones: es posible usar la acumulación para estimar historias, establecer y asignar tareas y marcar prioridades de iteraciones para ellas.

- Reuniones diarias: con el panel de control se puede obtener una vista profunda del progreso a fin de prepararse para reuniones diarias.

- Gráfico de trabajo pendiente: administra el progreso mediante el seguimiento del trabajo total restante en la iteración.

- Gráfico de velocidad: realiza un seguimiento de la velocidad del equipo y pronósticos precisos mediante el seguimiento de la cantidad de trabajo completado en cada iteración.

- Colaboración en equipo: comunica el progreso a los equipos locales y distribuidos. Además, comparte listas de tareas, comentarios y asignaciones.

- Métricas, informes y análisis ágiles: permiten realizar seguimiento y proyección de tiempo, informes de progreso fáciles de entender para los interesados, garantía de calidad y progreso con herramientas para identificar y eliminar los obstáculos del proyecto, evaluar el rendimiento y las finanzas.

- Integraciones: necesita complementos para ampliar la funcionalidad o al menos una API abierta.

Ventajas de la administración de proyectos ágiles

- **Mejora de la calidad del producto**: estas metodologías fomentan el enfoque proactivo de los miembros del equipo en la búsqueda de la excelencia del producto. Además, la integración, comprobación y mejora continua de las propiedades del producto mejoran considerablemente el resultado final.

- **Mayor satisfacción del cliente**: el cliente está más satisfecho al verse involucrado y comprometido a lo largo de todo el proceso de desarrollo. Mediante varias demostraciones y entregas, el cliente vive a tiempo real las mejoras introducidas en el proceso.

- **Mayor motivación de los trabajadores**: los equipos de trabajo autogestionados facilitan el desarrollo de la capacidad creativa y de innovación entre sus miembros.

- **Trabajo colaborativo**: la división del trabajo por distintos equipos y roles junto al desarrollo de reuniones frecuentes permite una mejor organización del trabajo.

- **Uso de métricas más relevantes**: las métricas utilizadas para estimar parámetros como tiempo, coste, rendimiento, etc. son normalmente más reales en los proyectos ágiles que en los tradicionales. Gracias a la división en pequeños equipos y fases podemos ser más conscientes de lo que está sucediendo.

- **Mayor control y capacidad de predicción**: la oportunidad de revisar y adaptar el producto a lo largo del proceso ágil permite a todos los miembros del proyecto ejercer un mayor control sobre su trabajo, cosa que permite mejorar la capacidad de predicción en tiempo y costes.

- **Reducción de costos**: la gestión ágil del proyecto elimina prácticamente la posibilidad de fracaso absoluto en el proyecto, porque los errores se van identificando a lo largo del desarrollo en

lugar de esperar a que el producto esté acabado y toda la inversión realizada. (Rosselló Villán, 2019, https://bit.ly/3oZmrX7)

A continuación, se presenta a la Empresa Sumar el desarrollo dirigido por un plan para que se pueda organizar.

1. Desarrollo dirigido por un plan

El desarrollo dirigido por un plan o basado en un plan es un enfoque para la ingeniería de ***software***, donde el proceso de desarrollo se planea a detalle.

2. Objetivo general

2.1. Apoyar la toma de decisiones del proyecto y medir el progreso.
2.2. Se sustenta en técnicas de administración de proyectos de ingeniería.

3. Planes de proyecto

3.1. Se establecen los recursos disponibles para el proyecto, la división del trabajo y un calendario para realizar el trabajo.

3.1.1. Cuestiones para identificar al realizar un plan

3.1.1.1. Riesgos para el proyecto.

3.1.1.2. El *software* en desarrollo.

3.1.1.3. Tipo de proyecto.

3.1.1.4. Organización.

3.1.2. Complementos del plan de proyecto

3.1.2.1. Ejemplos de complementos del plan de proyecto

3.1.2.1.1. Plan de calidad.

3.1.2.1.2. Plan de validación.

3.1.2.1.3. Configuración del plan de gestión.

3.1.2.1.4. Plan de mantenimiento.

3.1.2.1.5. Plan de desarrollo de personal.

3.1.2.2. Introducción

3.1.2.2.1. Los objetivos del proyecto y establecimiento de las restricciones.

3.1.2.3. Organización del proyecto

3.1.2.3.1. Forma en la que está organizado el equipo de desarrollo.

3.1.2.4. Análisis de riesgo

3.1.2.4.1. Detalla los posibles riesgos del proyecto, la probabilidad de que surjan dichos riesgos y las estrategias propuestas para reducir el riesgo.

3.1.2.5. Requerimientos de recursos de hardware y software

3.1.2.5.1. Detalle del *hardware* y el *software* de soporte requeridos para realizar el desarrollo.

3.1.2.6. División del trabajo

3.1.2.6.1. Establece la división del proyecto en actividades e identifica los plazos y las entregas asociados con cada actividad.

3.1.2.7. Calendario del proyecto

3.1.2.7.1. Indica las dependencias entre las actividades, el tiempo estimado requerido para alcanzar cada plazo y la asignación.

3.1.2.8. Mecanismos de monitorización y reporte

3.1.2.8.1. Son los informes administrativos que deben producirse, cuándo tienen que elaborarse y los mecanismos de monitorización del proyecto que se usarán.

4. El proceso de planeación

4.1. La planeación del proyecto es un proceso iterativo que comienza cuando se diseña un plan de proyecto inicial durante la fase de arranque del proyecto.

4.1.1. Las restricciones que afectan al proyecto

4.1.1.1. Fecha de entrega requerida.

4.1.1.2. Personal disponible.

4.1.1.3. Presupuesto global.

4.1.1.4. Herramientas disponibles.

4.1.1.5. Hay que identificar hitos.

4.1.1.5.1. Los hitos son puntos en el calendario contra los que se puede valorar el avance.

4.1.1.5.2. Entregables del proyecto.

4.1.1.5.2. Los entregables son productos de trabajo que se proporcionan al cliente.

4.1.2. El proceso entra en un ciclo

4.1.2.1. Se prepara un calendario estimado para el proyecto y se inician las actividades definidas en el calendario o se concede el permiso para continuarlas.

5. Calendarización de proyectos

5.1. Es el proceso de decidir cómo se organizará el trabajo en un proyecto como tareas separadas y cuándo y cómo se ejecutarán dichas tareas

5.1.1. Tareas

5.1.2. Duración de esperas.

5.1.3. Esfuerzo.

5.1.4. Dependencias entre las tareas.

5.1.5. Representación del calendario.

5.1.5.1. Los calendarios de proyecto pueden representarse simplemente en una tabla u hoja de cálculo que indique las tareas, el esfuerzo, la duración esperada y las dependencias entre las tareas.

5.1.5.1.1. Dos tipos de representación que se usan comúnmente.

5.1.5.1.1.1. Gráficas de barras basadas en el calendario. Señalan al responsable de cada actividad, el tiempo transcurrido previsto y la fecha en que se programó el inicio y el fin de la actividad.

5.1.5.1.1.2. Redes de actividad. Son diagramas de red que muestran las dependencias entre las diferentes actividades que constituyen un proyecto.

Finalmente, a la Empresa Sumar se le propone un plan de capacitación para que pueda enterarse de la estructura de la metodología ágil *scrum,* la metodología ágil XP y la metodología ágil *kanban*.

Metodología ágil scrum

Descubramos la historia de *scrum*

El concepto de *scrum* tiene su origen en un estudio de 1986 sobre los nuevos procesos de desarrollo utilizados en productos exitosos en Japón y los Estados Unidos (cámaras de fotos de Canon, fotocopiadoras de Xerox, automóviles de Honda, ordenadores de HP y otros). Los equipos que desarrollaron estos productos partían de requisitos muy generales, así como novedosos y debían salir al mercado en mucho menos tiempo del que se tardó en lanzar productos anteriores. Estos equipos seguían patrones de ejecución de proyecto muy similares. En este estudio se comparaba la forma de trabajo de estos equipos altamente productivos y multidisciplinares con la colaboración entre los jugadores de Rugby y su formación de *scrum*.

En 1993 se realizó el primer *scrum* para desarrollo de *software* y en 1995 el proceso fue formalizado. En 2001 un grupo de personas muy relevantes en lo que empezaba a ser el desarrollo ágil escribieron los valores fundamentales de los procesos ágiles.

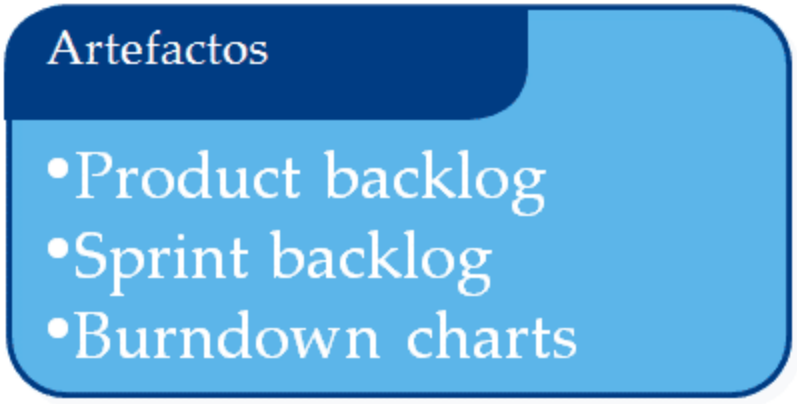
Desde 1995 miles de proyectos en todo el mundo han utilizado *scrum* para el desarrollo de productos, tanto en empresas pequeñas, con tan solo 3 personas desarrollando un producto, como en multinacionales. (Scrum, s. f. b, https://bit.ly/3fvV8Rc)

Organización de scrum

1 Artefactos

Encontramos los artefactos presentes en la figura 1.

Figura 1: Artefactos



Fuente: Grafeuille, 2008, https://bit.ly/3fsiCqs

1.1. Product backlog

El *product backlog* es un documento de alto nivel para todo el proyecto. Contiene descripciones genéricas de todos los requerimientos y funcionalidades deseables. Además, en él se realiza la lista de requerimientos para trabajar en el *scrum*, es abierto y cualquiera puede modificarlo.

Figura 2: Ejemplo de *product backlog*

Fuente: elaboración propia con base en Grafeuille, 2008, https://bit.ly/3fsiCqs

Backlog item	Estimación
Permitir que un invitado haga una reserva.	3
Como invitado, quiero cancelar la reserva.	5
Como invitado, quiero cambiar las fechas de una reserva.	3
Como empleado de hotel, puedo ejecutar informes de los ingresos por habitación disponible.	8
Mejorar el manejo de excepciones.	8
...	30
...	50

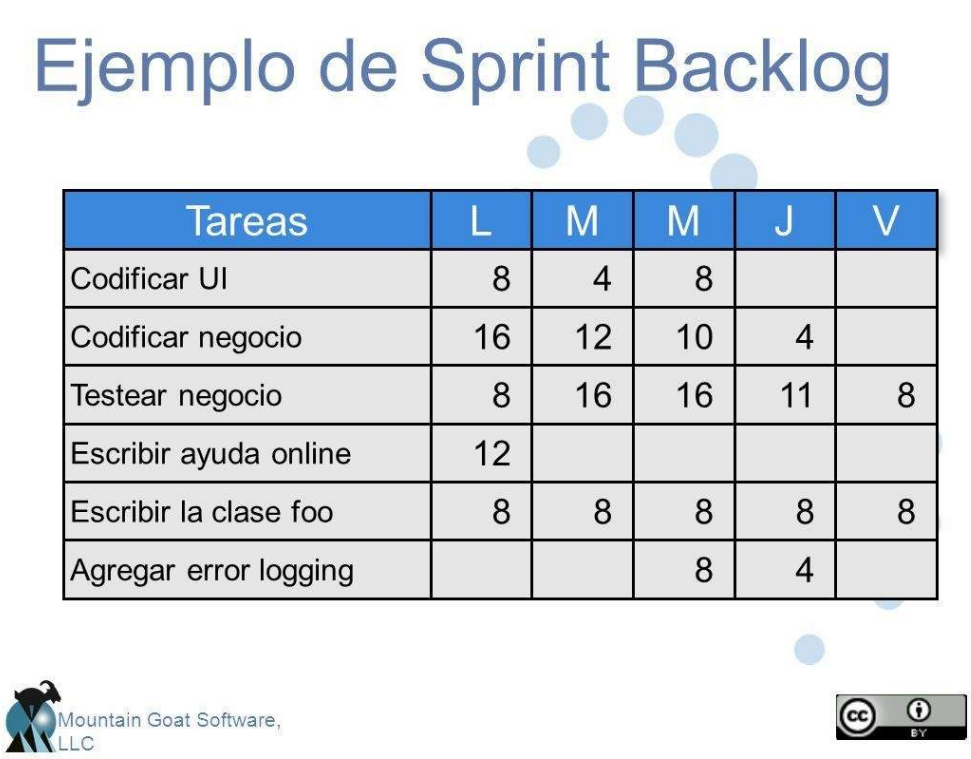
1.2. Sprint backlog

Es el documento donde se establece y compromete el equipo a cumplir los requerimientos funcionales durante el desarrollo del *sprint*. El *sprint* dura de 2 a 4 semanas.

En el ejemplo se pueden observar las tareas que se comprometen a realizar durante el *sprint* y la duración de cada una.

Figura 3: Ejemplo de *sprint backlog*

Fuente: Grafeuille, 2008, https://bit.ly/3fsiCqs



1.3. Burndowns charts

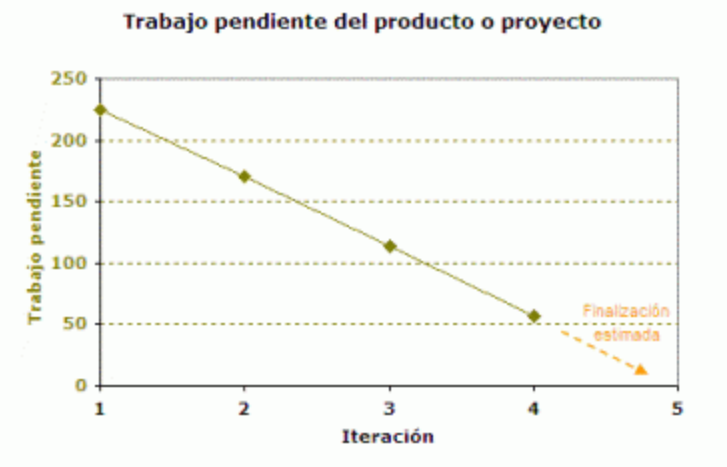
Sobre el eje horizontal se muestran los ***sprint*** que se desarrollan. Sobre el eje vertical, los requisitos que faltan completar al final de cada ***sprint***.

En la figura 4 podemos ver un ejemplo de ***burndowns charts.***

En el ***burndown chart*** se realiza el seguimiento diario sobre cómo se van desarrollando los requerimientos funcionales para el desarrollo del proyecto.

Figura 4: ejemplo de *burndowns charts*

Fuente: [imagen sin título sobre ejemplo de ***burndowns charts***], 2015, https://bit.ly/3bZwA0X



2 Roles

2.1. Product owner

- Define las funcionalidades del producto.
- Define el diseño del producto.
- Representa al cliente.
- Prioriza la funcionalidad de acuerdo con el valor del negocio.
- Acepta o rechaza resultados del trabajo en equipo.
- Vuelca lo que hay que hacer en el trabajo.

2.2. El team o equipo

- En general, se conforma de 5 a 9 integrantes.
- Auto organizado.
- Realiza tareas multifuncionales.
- Solo puede haber cambios cuando termina el sprint.
- Miembros ***full time***.

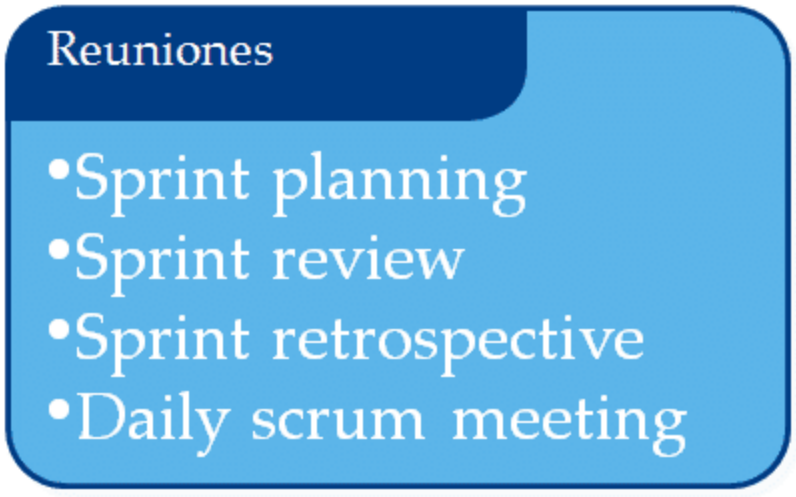
2.3. Scrum master

- Durante la iteración, el facilitador (***scrum master***) se encarga de que el equipo pueda cumplir con su compromiso y no merme su productividad.
- Elimina los obstáculos que el equipo no puede resolver por sí mismo.
- Protege al equipo de interrupciones externas que puedan afectar su compromiso o productividad.
- Protege al equipo.
- No es un ***leader project.***
- Representa la gestión del proyecto.
- Resuelve impedimentos.
- Asegura un equipo funcional y productivo.
- Responsable de promover los valores y prácticas de ***scrum.***
- Se asegura de que el equipo es completamente funcional y productivo.
- Permite la estrecha cooperación en todos los roles y funciones.

- Escudo del equipo de interferencias.

3 Reuniones

Figura 5: Reuniones



Fuente: Grafeuille, 2008, https://bit.ly/3fsiCqs

3.1. Sprint planning meeting (reunión de planificación del sprint)

- Es la reunión de planificación del *sprint (sprint planning meeting)*.
- Al inicio del ciclo *sprint* (cada 15 o 30 días), una reunión de planificación del *sprint* se lleva a cabo.

- Selecciona el trabajo a realizar.
- Interviene entre el ***product owner*** y el equipo que cumplirá los requerimientos. Estos quedan detallados en el ***sprint backlog.***
- Los requerimientos se extraen del ***product backlog.***
- Prepara con el equipo completo el ***sprint backlog,*** el cual detalla el tiempo que tomará hacer el trabajo.
- Identifica y comunica cuánto del trabajo es probable que se realice durante el actual ***sprint.***
- **8 horas como límite.**

3.2. El sprint

- El ***sprint*** es el período en el cual se lleva a cabo el trabajo en sí.
- Es recomendado que la duración de los ***sprint*** sea constante y definida por el equipo de acuerdo con su propia experiencia.
- Se puede comenzar con una duración de ***sprint*** en particular (2 o 4 semanas) e ir ajustándolo de acuerdo con el ritmo del equipo.
- Al final de cada ***sprint,*** el equipo deberá presentar los avances logrados y entregar un producto con características propuestas por el cliente.
- Asimismo, **se recomienda no cambiar los objetivos del *sprint backlog* a menos que la falta de estos cambios amenace al éxito del proyecto.**
- La constancia hace a la concentración y mejor productividad del equipo de trabajo.

3.3. Daily scrum (reunión diaria)

- Duración 15 minutos.
- Todos los integrantes se mantienen de pie para evitar pérdida de tiempo y desconcentración.
- Preguntas: ¿qué has hecho ayer? ¿Qué es lo que estás planeando hacer hoy? ¿Has tenido algún problema que te haya impedido alcanzar el objetivo?

3.3.1 Roles en el *daily scrum*

En la reunión nos encontramos con los roles de cerdo y gallinas. El cerdo se compromete con el proyecto, la gallina pone el huevo y se va. Se suele tener un castigo para el que llega tarde, por ejemplo, se le cuelga una gallina en el pecho.

Todos son bienvenidos, pero solo los cerdos pueden hablar. La reunión tiene una duración fija de 15 minutos, de forma independiente del tamaño del equipo. Todos los asistentes deben mantenerse de pie (esto ayuda a mantener la reunión corta). La reunión debe ocurrir en la misma ubicación y a la misma hora todos los días.

Como mencionamos anteriormente, durante la reunión, cada miembro del equipo contesta a tres preguntas: ¿qué has hecho desde ayer? ¿Qué es lo que estás planeando hacer hoy? ¿Has tenido algún problema que te haya impedido alcanzar tu objetivo? (es el papel del ***scrum master*** recordar estos impedimentos).

Asumen roles de cerdo:

- *scrum master* (facilitador);

- *product owner* (es el dueño del producto y representa al cliente);
- equipo (realiza las tareas).

Asumen roles de gallina:

- usuario;
- gerente.

3.4. Scrum de scrum (revisión de todos los equipos de las reuniones diarias)

- Se realiza cada día, normalmente, después del ***daily scrum.***
- Estas reuniones permiten a los grupos de equipos discutir su trabajo, enfocándose especialmente en áreas de solapamiento e integración.
- Asiste una persona asignada por cada equipo.

La agenda será la misma que la del ***daily scrum,*** solo se añadirán las siguientes cuatro preguntas.

- ¿Qué ha hecho tu equipo desde la última reunión?
- ¿Qué hará tu equipo antes de que nos volvamos a reunir?
- ¿Existe algo de demora o estorbo en tu equipo?
- ¿Estás a punto de poner algo en el camino de otro equipo?

Revisión del *sprint (sprint review meeting)*

- Se realiza cuando termina cada ***sprint.***
- Revisar el trabajo que fue completado y no completado.

- Presentar el trabajo completado a los interesados.
- El trabajo incompleto no puede ser demostrado.
- Cuatro horas como límite.
- Se empieza a planificar el próximo *sprint*.

Interviene:

- el equipo;
- el cliente;
- el *product owner*.

4 Descubramos la potencialidad de un equipo

Tenemos un Sprint que dura dos semanas, días hábiles de 8 horas de lunes a viernes. Entonces, 8 horas x 10 días = 80 horas.

Asimismo, intervienen 3 personas: Juan, María y José. Si José falta un día, tiene 8 horas menos. Se demuestra esto en la tabla 1.

Tabla 1: Ejemplo de la potencialidad de un equipo

Actor	Total	Presente

Actor	Total	Presente
Juan	80	80
María	80	80
José	80	72
Total Potencialidad	240	**232**

Fuente: elaboración propia.

De esta manera, se puede observar que el equipo tuvo una potencialidad de 232 horas debido a que José faltó un día.

Metodología ágil XP *(extreme programing)*

Utiliza desarrollo iterativo y participación del cliente a niveles extremos.

CARACTERÍSTICAS

- "En la programación extrema todos los requerimientos se expresan como escenarios llamados historias de usuario, los cuales son implementados

directamente como una serie de tareas" (Escuela Especializada en Ingeniería ITCA, s. f., https://bit.ly/3wGg3Xv).

- Se propone que los desarrolladores trabajen en pareja. Se desarrollan pruebas para cada tarea antes de escribir el código.
- "Todas las pruebas se deben ejecutar satisfactoriamente cuando el código nuevo se integre al sistema" (Escuela Especializada en Ingeniería ITCA, s. f., https://bit.ly/3wGg3Xv).

Figura 6: Pasos para el desarrollo iterativo



Fuente: adaptación propia con base en Escuela Especializada en Ingeniería ITCA, s. f.

Tabla 2: Descripción de los principios o prácticas

Principio o práctica	Descripción
Planificación incremental	Los requerimientos se registran en tarjetas de historias y las historias a incluir en una entrega se determinan según el tiempo disponible y su prioridad relativa. Los desarrolladores dividen estas Historias en «Tareas» de desarrollo. Véanse las Figuras 17.6 y 17.7.
Entregas pequeñas	El mínimo conjunto útil de funcionalidad que proporcione valor de negocio se desarrolla primero. Las entregas del sistema son frecuentes e incrementalmente añaden funcionalidad a la primera entrega.
Diseño sencillo	Sólo se lleva a cabo el diseño necesario para cumplir los requerimientos actuales.
Desarrollo previamente probado	Se utiliza un sistema de pruebas de unidad automatizado para escribir pruebas para nuevas funcionalidades antes de que éstas se implementen.
Refactorización	Se espera que todos los desarrolladores refactoricen el código continuamente tan pronto como encuentren posibles mejoras en el código. Esto conserva el código sencillo y mantenible.
Programación en parejas	Los desarrolladores trabajan en parejas, verificando cada uno el trabajo del otro y proporcionando la ayuda necesaria para hacer siempre un buen trabajo.

Programación en parejas	Los desarrolladores trabajan en parejas, verificando cada uno el trabajo del otro y proporcionando la ayuda necesaria para hacer siempre un buen trabajo.
Propiedad colectiva	Las parejas de desarrolladores trabajan en todas las áreas del sistema, de modo que no desarrollen islas de conocimientos y todos los desarrolladores posean todo el código. Cualquiera puede cambiar cualquier cosa.
Integración continua	En cuanto acaba el trabajo en una tarea, se integra en el sistema entero. Después de la integración, se deben pasar al sistema todas las pruebas de unidad.
Ritmo sostenible	No se consideran aceptables grandes cantidades de horas extras, ya que a menudo el efecto que tienen es que se reduce la calidad del código y la productividad a medio plazo.
Cliente presente	Debe estar disponible al equipo de la XP un representante de los usuarios finales del sistema (el cliente) a tiempo completo. En un proceso de la programación extrema, el cliente es miembro del equipo de desarrollo y es responsable de formular al equipo los requerimientos del sistema para su implementación.

Fuente: Sommerville, 2005, p. 365.

ROLES DE XP

Los roles de acuerdo con la propuesta original de Beck son los siguientes.

- **Programador:** el programador escribe las pruebas unitarias y produce el código del sistema.
- **Cliente:** escribe las historias de usuario y las pruebas funcionales para validar su implementación. Además, asigna prioridad a las historias de usuario y decide cuáles se implementan en cada iteración, centrándose en las que aportan mayor valor al negocio.

- **Encargado de pruebas (*tester*):** ayuda al cliente a escribir las pruebas funcionales. Ejecuta las pruebas regularmente, difunde los resultados en el equipo y es responsable de las herramientas de soporte para pruebas.
- **Encargado de seguimiento (*tracker*):** proporciona retroalimentación al equipo. Verifica el grado de acierto entre las estimaciones realizadas y el tiempo real dedicado para mejorar futuras estimaciones. Realiza el seguimiento del progreso de cada iteración.
- **Entrenador (*coach*):** es responsable del proceso global. Debe proveer guías al equipo para que se apliquen las prácticas XP y se siga el proceso correctamente.
- **Consultor:** es un miembro externo del equipo con un conocimiento específico en algún tema necesario para el proyecto y en el cual pueden surgir problemas.
- **Gestor *(big boss)*:** es el vínculo entre clientes y programadores, ayuda a que el equipo trabaje efectivamente al crear las condiciones adecuadas. Su labor esencial es la coordinación.

Diferencia entre metodologías

Tabla 3: Diferencia entre metodologías ágiles y no ágiles

Metodologías Ágiles	Metodologías Tradicionales
Basadas en heurísticas provenientes de prácticas de producción de código	Basadas en normas provenientes de estándares seguidos por el entorno de desarrollo
Especialmente preparados para cambios durante el proyecto	Cierta resistencia a los cambios
Impuestas internamente (por el equipo)	Impuestas externamente
Proceso menos controlado, con pocos principios	Proceso mucho más controlado, con numerosas políticas/normas
No existe contrato tradicional o al menos es bastante flexible	Existe un contrato prefijado
El cliente es parte del equipo de desarrollo	El cliente interactúa con el equipo de desarrollo mediante reuniones
Grupos pequeños (<10 integrantes) y trabajando en el mismo sitio	Grupos grandes y posiblemente distribuidos
Pocos artefactos	Más artefactos
Pocos roles	Más roles
Menos énfasis en la arquitectura del software	La arquitectura del software es esencial y se expresa mediante modelos

Tabla 1. Diferencias entre metodologías ágiles y no ágiles

Fuente: [imagen sin título sobre diferencias entre metodologías ágiles y no ágiles], s. f., https://bit.ly/2RQEZg4

Metodología ágil *kanban*

Kanban proviene del *kanji*, donde *kan-* significa visual y *-ban* significa tarjeta o tablero. Es un concepto de producción justo a tiempo (JIT). El *kanban* es una tarjeta física que se utiliza en el sistema de producción para soportar un control productivo descentralizado por demanda.

Además, *kanban* es una herramienta proveniente de la filosofía *lean* de tipo *pull*, lo que significa que los recursos deciden cuándo y cuánto trabajo se comprometen a realizar. *Kanban* refleja a través de un tablero la situación de cada tarea del proyecto, con una visualización comprensible para todo el

mundo. Permite a la gente pensar por sí misma y sacar las expectativas razonables sin necesidad de una interpretación excesiva por escrito del estado (Castillo Ali, 2019). Los siguientes elementos son esenciales.

***Stream*:** visualizar el flujo real de trabajo a través de los equipos y la comprensión de lo que están haciendo en cada paso. El tablero de *kanban* consiste en una serie de columnas que representan las diferentes etapas de trabajo.

Contenido: representar los diferentes tipos de trabajo y asegurarse de que todo el trabajo se visualiza de alguna manera que sea comprensible para todos los involucrados. Normalmente, esto se hace con las tarjetas que se colocan en las columnas definidas anteriormente.

Límites: definir claramente los límites de la cantidad de trabajo que el equipo puede soportar dentro de cada etapa del *stream*. Estos límites denotan el punto a partir del cual un equipo no logra progresos adicionales al tratar de tomar un trabajo adicional. Estos límites se definen como el número de tarjetas permitidas en cada columna.

Políticas: estas políticas toman la forma de declaraciones comprobables que declaran en forma explícita y específica qué

> significa avanzar una tarea (tarjeta) de una columna a la siguiente. Las políticas ayudan a crear confianza en la visualización de todos los interesados. (Castillo Ali, 2019, p. 32)

Trabajo en kanban

1. Ve el *workflow:*

- divide el trabajo en piezas y escribe cada una de ellas en tarjetas que se colocan en el tablero (***kanban board***);
- utiliza columnas con nombres para ilustrar dónde se encuentra dentro del proceso o ***workflow*** cada ítem o tarea.

2. Limita el *work in progress* (WIP): asigna límites específicos de cuántos ítems pueden ser procesados a la vez dentro de cada columna del ***workflow***.

3. Mide el *lead time* (ciclo de tiempo): es el tiempo promedio para completar un ítem. En otras palabras, mide el tiempo que el ítem tarda en pasar por todas las columnas del ***workflow*** hasta llegar al final. El ***kanban*** tiene por objetivo optimizar el proceso para hacer el ***lead time*** lo más pequeño y predecible que se pueda. (Castillo Ali, 2019, p. 34)

Lo único que *kanban* prescribe es que el *workflow* pueda verse y ser comprendido por todos. El propósito es crear un flujo continuo a través del sistema y minimizar el *lead time* o *cycle time*.

> Cada columna representa un estado dentro del *workflow*, o un *buffer* entre dos estados del *workflow*. La cantidad de columnas la determina cada uno de acuerdo con su experiencia y necesidad. Cuando se ha llegado al límite *kanban* de una columna y no se tiene otra cosa que hacer, comience a buscar los cuellos de botella y solucionarlos. Si no existen es un indicativo que el límite podría ser demasiado bajo. Si se nota que los ítems permanecen en una columna por largo tiempo, es un indicativo de que el límite es demasiado alto. La regla es la siguiente:
>
> - límite *kanban* demasiado bajo = recursos desperdiciados = baja productividad;
> - límite *kanban* demasiado alto = tareas en cola = erróneo *lead time*. (Castillo Ali, 2019, p. 35)

Beneficios de kanban

- La visibilidad de los cuellos de botella ***(bottlenecks)*** en tiempo real. Esto permite al equipo que colabore para optimizar la cadena de valor en lugar de ocuparse cada uno de su parte. El síntoma de un cuello de botella es cuando vemos que la columna X está repleta de ítems mientras que la columna que sigue se encuentra vacía.
- Permite una evolución más gradual para pasar de ***waterfall*** a un desarrollo ágil de ***software***, para aquellas empresas que aún no se animan a intentar dar ese paso.
- Con ***kanban*** no se usan iteraciones. La realidad es que las iteraciones son opcionales.
- Con ***kanban*** no hay que estimar. La realidad es que las estimaciones son opcionales también.

  Para tener una idea clara sobre la estimación y velocidad en ***kanban*** repasemos muy brevemente lo relacionado con ***scrum.***

  Uno de los aspectos únicos de un proyecto ágil es que la carga de trabajo para cada iteración se determina al comienzo de cada iteración. Solo hay necesidad de planificar para la iteración actual y no una planificación anticipada pesada y completa como el método tradicional. Esto permite que el proyecto ágil sea mucho más flexible.

  Al comienzo de cada iteración se realiza una reunión entre ***product owner*** y el equipo del proyecto para determinar la carga de trabajo para la nueva iteración. Durante la reunión, se evalúa la cartera de requisitos y se selecciona la siguiente serie de casos de uso (o historias de usuario) que son de más alta prioridad. El nivel de esfuerzo para cada caso de uso debería

haber sido asignado cuando se añadió al ***backlog.*** Allí la estimación no debe ser horas de esfuerzo, sino más bien una serie de ***story points***.

Los ***story points*** son números arbitrarios utilizados para estimar el tamaño relativo de un caso de uso. La escala exacta es diferente de un equipo a otro. Por ejemplo, si un caso de uso tiene 10 puntos, otro caso de uso que es dos veces más complejo debería tener 20 puntos. La estimación total de los puntos incluye el tiempo de análisis, diseño, construcción, prueba e implementación.

Durante la reunión de planificación, el equipo del proyecto toma tantos ***story points*** como los que pueda completar dentro de la iteración, dentro de los más priorizados. Por supuesto, las prioridades pueden cambiar durante el proyecto.

Cuando esto sucede en un proyecto ágil, el cambio de prioridades es simplemente una cuestión de reasignar el orden de los casos de uso en el ***backlog.***

Es importante que el equipo del proyecto pueda determinar rápidamente la cantidad de trabajo que puede completar en cada iteración. Esto permitirá que la carga de trabajo se mantenga relativamente uniforme de iteración a iteración. Si el equipo del proyecto considera que no fue capaz de completar una serie de casos de uso en la iteración anterior, se ponen de acuerdo para tomar menos trabajo en la siguiente iteración. Del mismo modo, si el equipo se da cuenta de que podían haber hecho más trabajo en una iteración, deberían tomar más trabajo en la siguiente iteración. Veamos un ejemplo.

Digamos que el equipo inicia el proyecto y estima que pueden completar 40 puntos en una iteración. Sin embargo, a medida que avanza la iteración, se encuentran con que tienen que trabajar muchas horas extras y que no pueden completar todo el trabajo durante la iteración.

En la siguiente iteración, se decide entonces escoger solo 32 ***story points***. El equipo tiene éxito, pero durante la revisión final de la iteración se dan cuenta de que podrían haber hecho más. En la siguiente iteración deciden tomar 36 puntos de historia. Después de la iteración el equipo decide que este es un ritmo exigente pero totalmente factible para ellos. Por lo tanto, a partir de entonces el equipo toma 36 puntos de historia de casos de uso por iteración.

Este ritmo al que el equipo puede completar los puntos de historia del ***backlog*** se conoce como la velocidad del equipo. Los casos de uso que son seleccionados para una iteración necesitan ser completados en esa iteración. En un proyecto ágil es importante mantenerse en un ciclo de repetición constante. Si el caso de uso no está listo cuando la iteración está lista para pasarla a producción, el código de dicho caso de uso debe ser retirado para que el resto de la iteración pueda ser liberada a tiempo. No hay retrasos en la fecha de finalización de la iteración. El equipo se centra en completar todos los ***story points*** en la fecha comprometida una y otra vez. Este ritmo constante en cada iteración también se conoce como el ritmo del equipo o cadencia. (Castillo Ali, 2019, p. 38)

En *kanban*, la estimación no está prescrita. Por lo tanto, algunos equipos eligen hacer estimaciones y medir la velocidad al utilizar un tamaño de los requerimientos funcionales a cumplir: S, M, L, XL.

Haremos un repaso de los principales puntos de la lectura.

1) La gestión ágil de proyectos es un enfoque secuencial para planificar y guiar los procesos del proyecto.

- ◯ Verdadero.
- ◯ Falso.

SUBMIT

2) La administración ágil de un proyecto permite mantener los costos estables.

- ◯ Verdadero.

Falso.

SUBMIT

3) En la metodología ágil *scrum* existe una herramienta gráfica que permite ver el cumplimiento de los requerimientos funcionales una vez terminado el *sprint*. Debes identificar la herramienta en la siguiente lista.

El *burdown chart* permite ver el cumplimiento de los requerimientos funcionales una vez terminado el *sprint*.

El *product backlog* permite ver el cumplimiento de los requerimientos funcionales una vez terminado el *sprint*.

El *sprint backlog* permite ver el cumplimiento de los requerimientos funcionales una vez terminado el *sprint*.

El *product owner* permite ver el cumplimiento de los requerimientos funcionales una vez terminado el *sprint*.

El *scrum master* permite ver el cumplimiento de los requerimientos funcionales una vez terminado el *sprint*.

SUBMIT

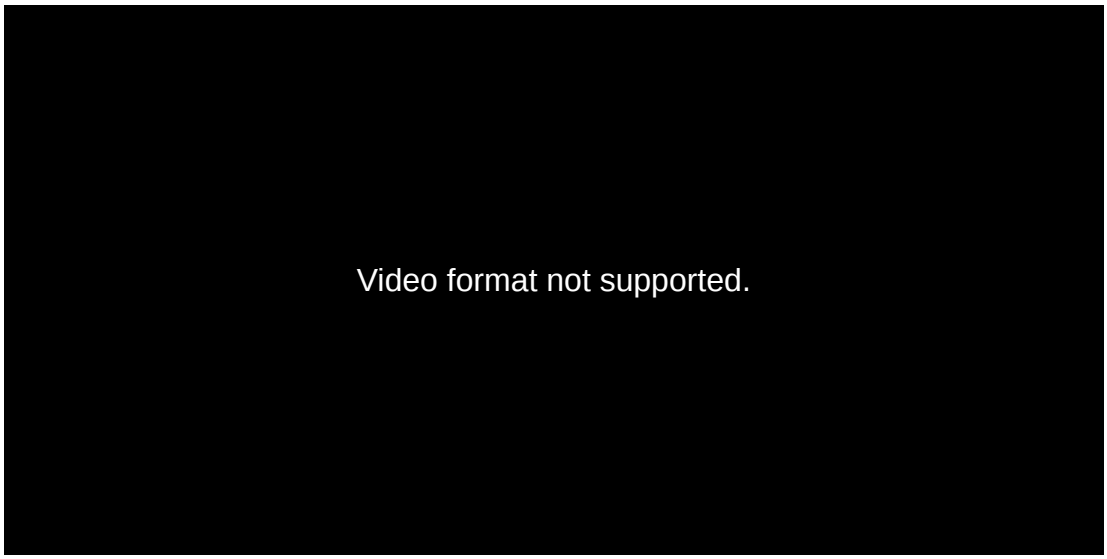
4) En la metodología XP hay principios que se deben cumplir. Identifica a continuación los principios que consideres necesarios que se cumplan.

- [ ] Programación en parejas.
- [ ] Refactorización.
- [ ] Integración continua.
- [ ] Diseño secuencial e incremental.
- [ ] Desarrollo secuencial e incremental.

SUBMIT

CONTINUAR

Video conceptual



CONTINUAR

Referencias

[Imagen sin título sobre diferencias entre metodologías ágiles y no ágiles], (s. f.). Recuperado de https://i.pinimg.com/originals/86/7c/73/867c73196ee6b63a2f7d9ec95eee54b8.png

[Imagen sin título sobre ejemplo de *burndowns charts*], (2015). Recuperado de https://scrumenespanol.files.wordpress.com/2015/09/proyectos-agiles-burndown-chart-alfa-20090930.gif?w=470&h=303

Castillo Ali, R. (2019). I*mplementación de kanban para la reducción de costos de consumo de combustible*. Recuperado de https://es.calameo.com/read/00442236325aa3dd790ea

Escuela Especializada en Ingeniería ITCA, (s. f.). Programación externa. En *Selección de técnicas de ingeniería de software*. Recuperado de https://virtual.itca.edu.sv/Mediadores/stis/42___programacin_extrema.html

Grafeuille, E. (2008). *Una introducción a scrum*. Recuperado de https://slidetodoc.com/una-introduccin-a-scrum-ernesto-grafeuille-noviembre-2008-2/

Rosselló Villán, V. (2019). Las metodologías ágiles más utilizadas y sus ventajas dentro de la empresa. En *IEBS*. Recuperado de https://www.iebschool.com/blog/que-son-metodologias-agiles-agile-scrum/

Scrum, (s. f. a). Desarrollo iterativo e incremental. En *Proyectos ágiles.org*. Recuperado de https://proyectosagiles.org/desarrollo-iterativo-incremental/

Scrum, (s. f. b). Historia de *scrum*. En Proyectos *ágiles.org*. Recuperado de https://proyectosagiles.org/historia-de-scrum/

Sommerville, I. (2005). *Ingeniería de software* (7.° ed.). Madrid, España: Editorial Pearson. Recuperado de https://www.academia.edu/15059886/Ingenieria_de_Software_Ian_Sommerville_7a_Edicion

CONTINUAR